

# *Object-oriented programming in C++*

## *Memory*

Dr. Claudia Weis, Faculty of Computer Science

Fakultät für Informatik

UNIVERSITÄT DUISBURG ESSEN

*Offen im Denken*

# *Where is 5 stored?*

```
3    int x = 5;
```

# *Fundamentals*
## *Why we talk about memory*

Fakultät für Informatik

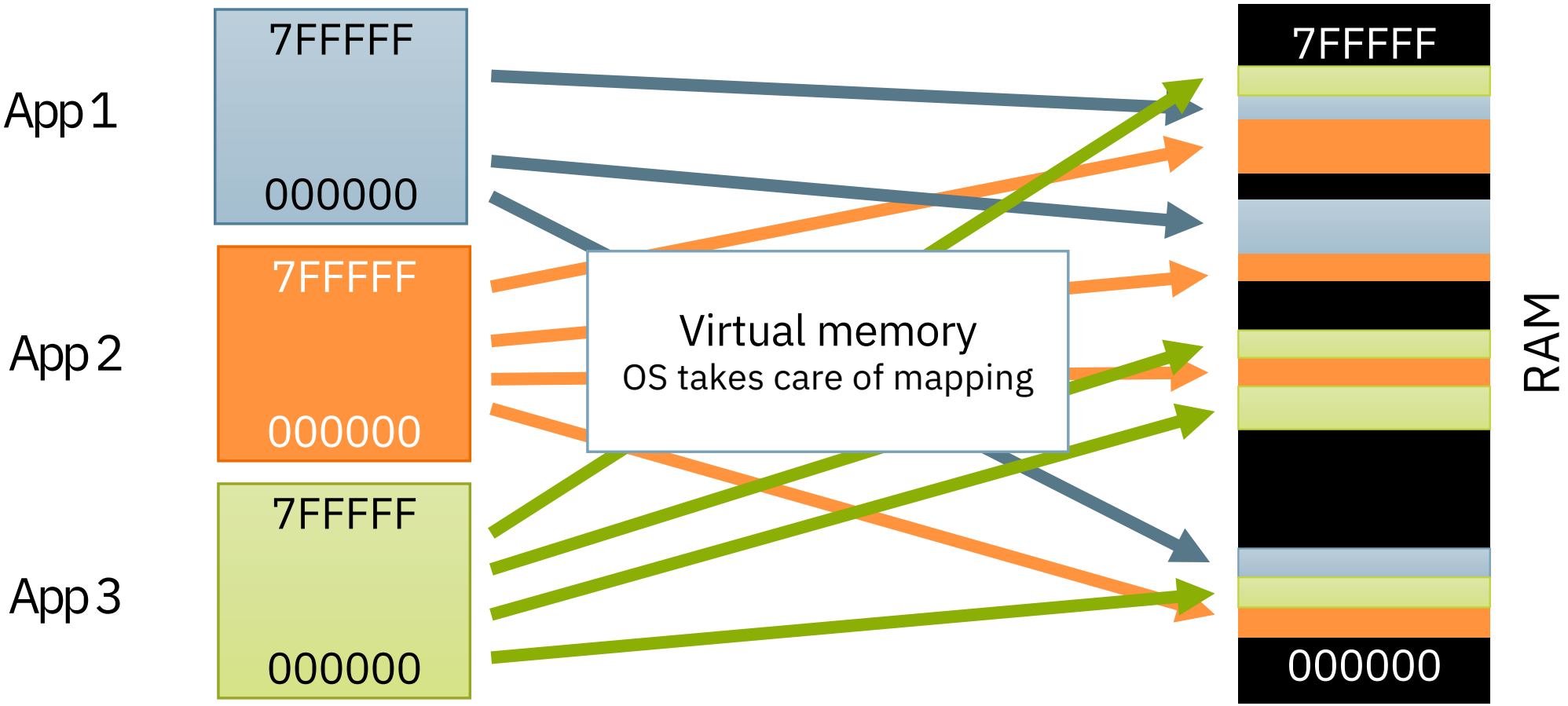
UDE

- For your application it looks like it is the only program running on the computer
- The operating system manages each application's memory.
- Virtual Memory serves as a layer of indirection.

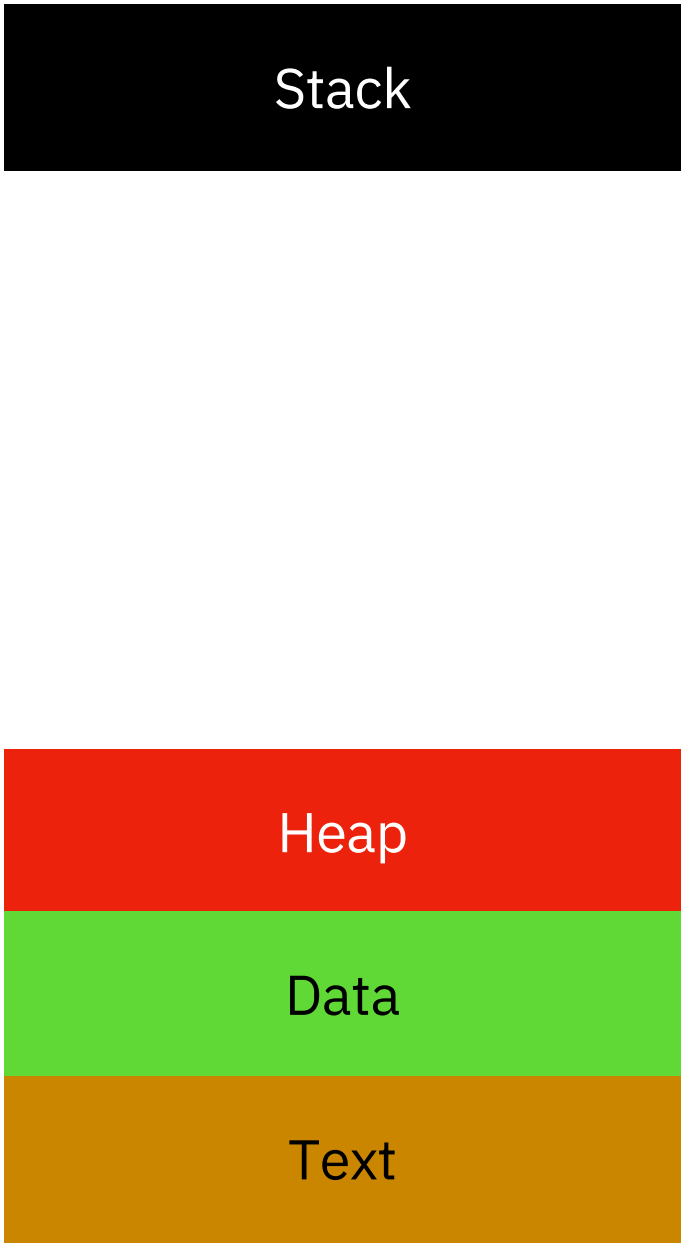
- There are different memory regions.
  - Each works fundamentally different.

  - Knowing where your data goes will improve the way you think about your programmes.

# *Virtual memory*

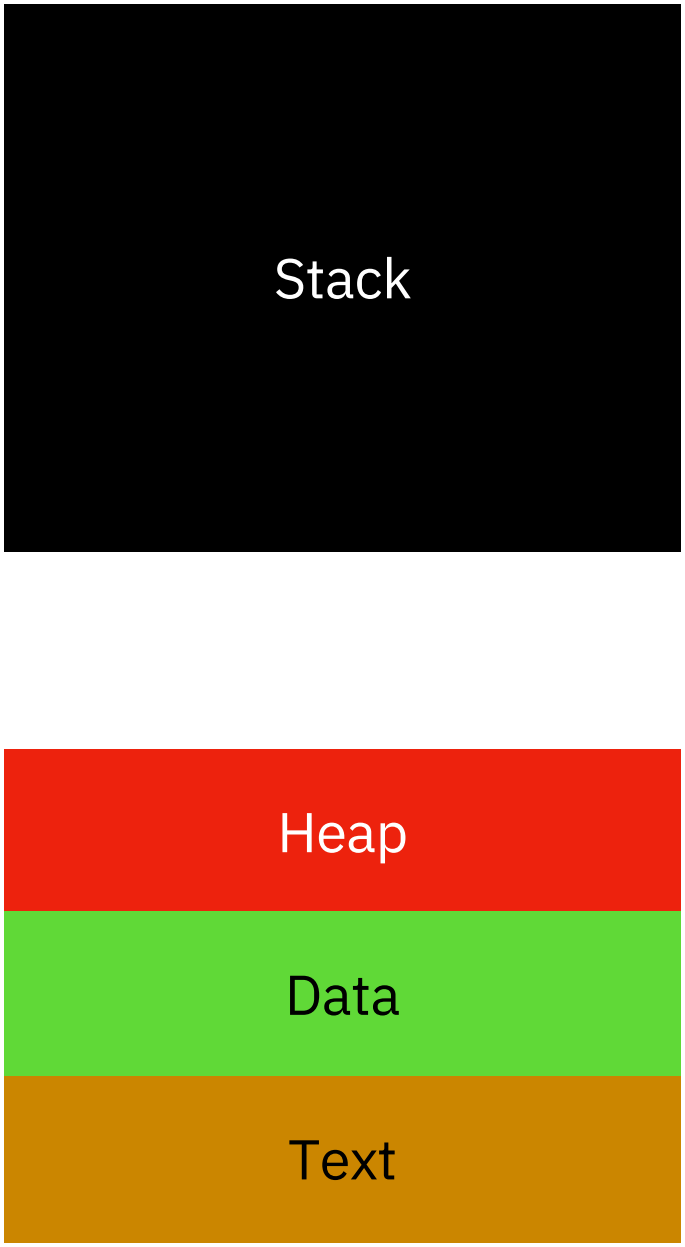


# *Process memory layout*

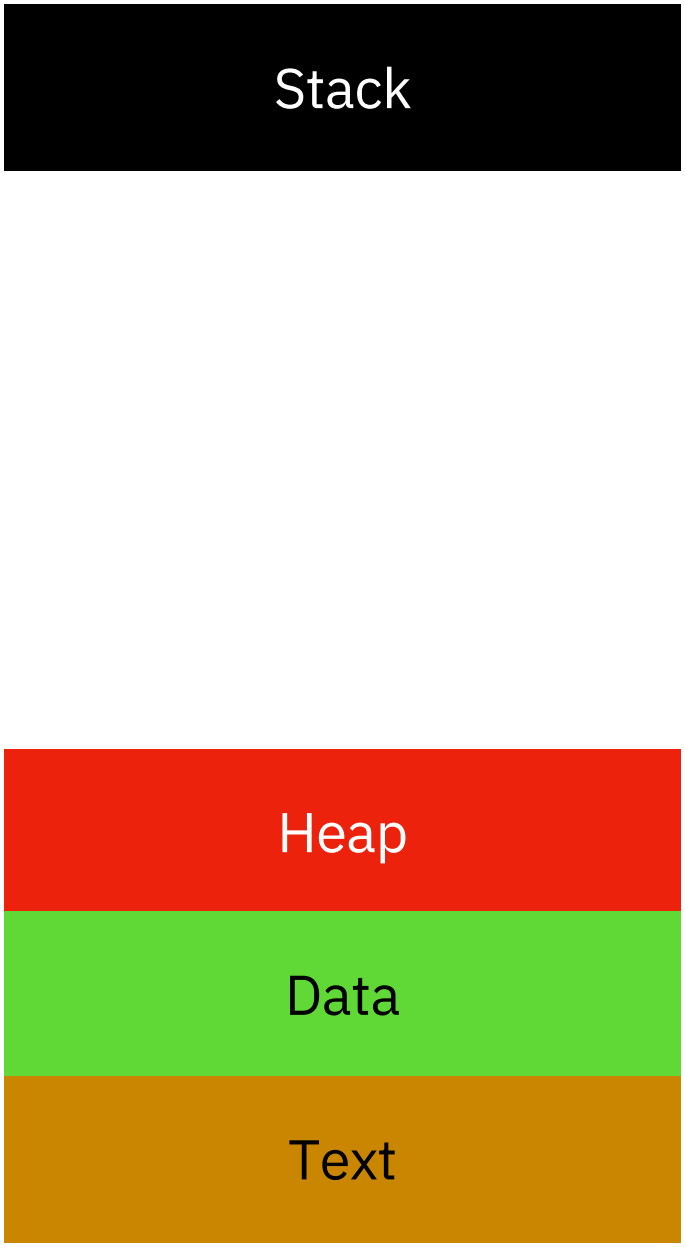


# *What goes to the stack*

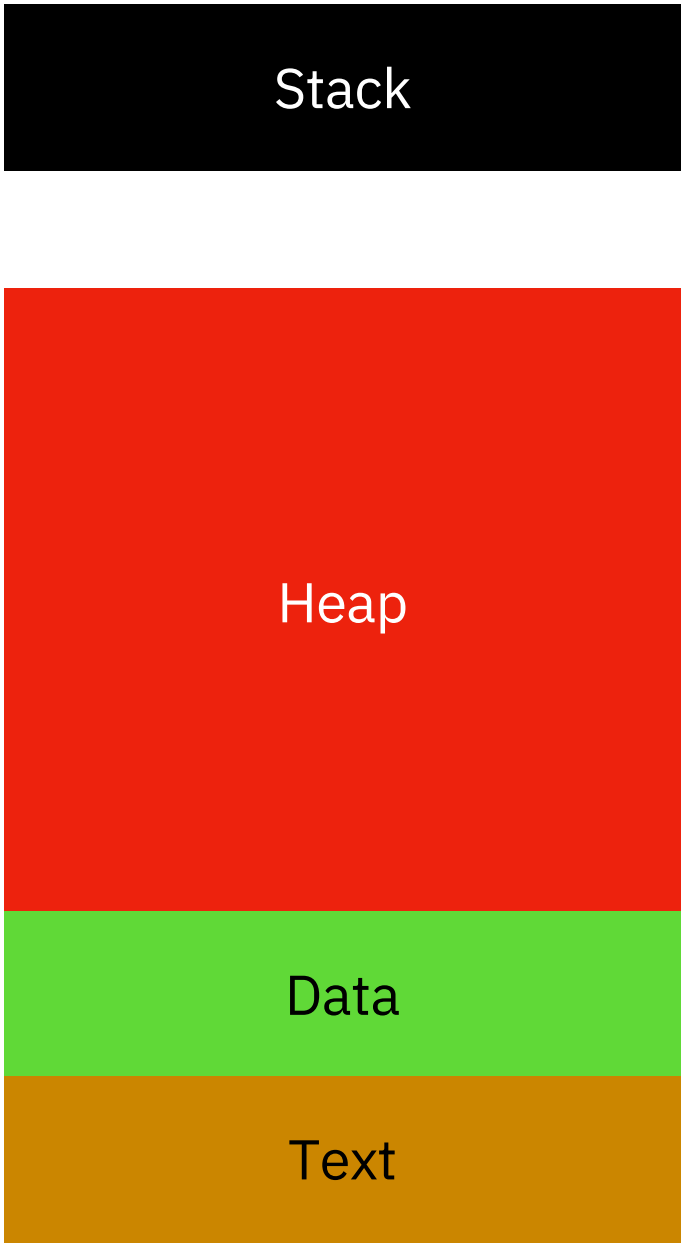
Calling functions
Local variables
➢ Dynamic memory on stack



*Stack is freed when functions return*



# *What goes to the heap*

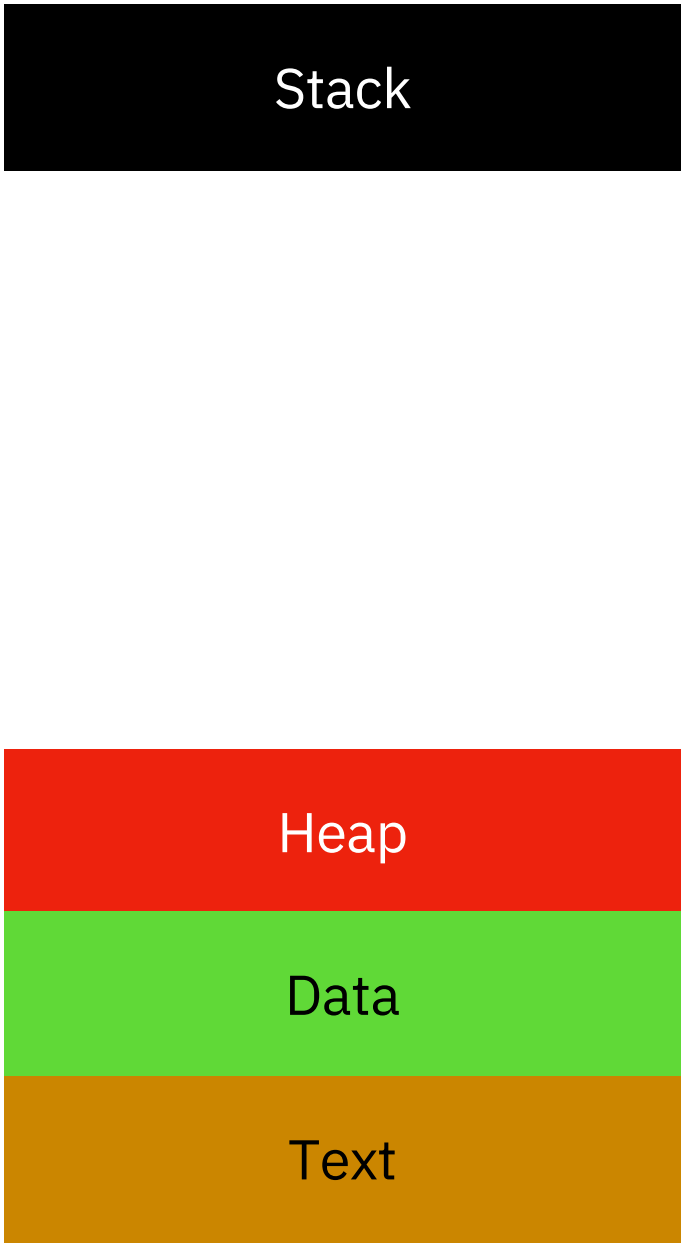



Detached from program flow
Filled and freed independent of function calls and returns

➢ Dynamic memory on heap

Old style: manually requested (using new/delete or malloc/free)
Modern C++ has mechanisms to care for it. We talk about that later…

*Heap is freed “manually”*

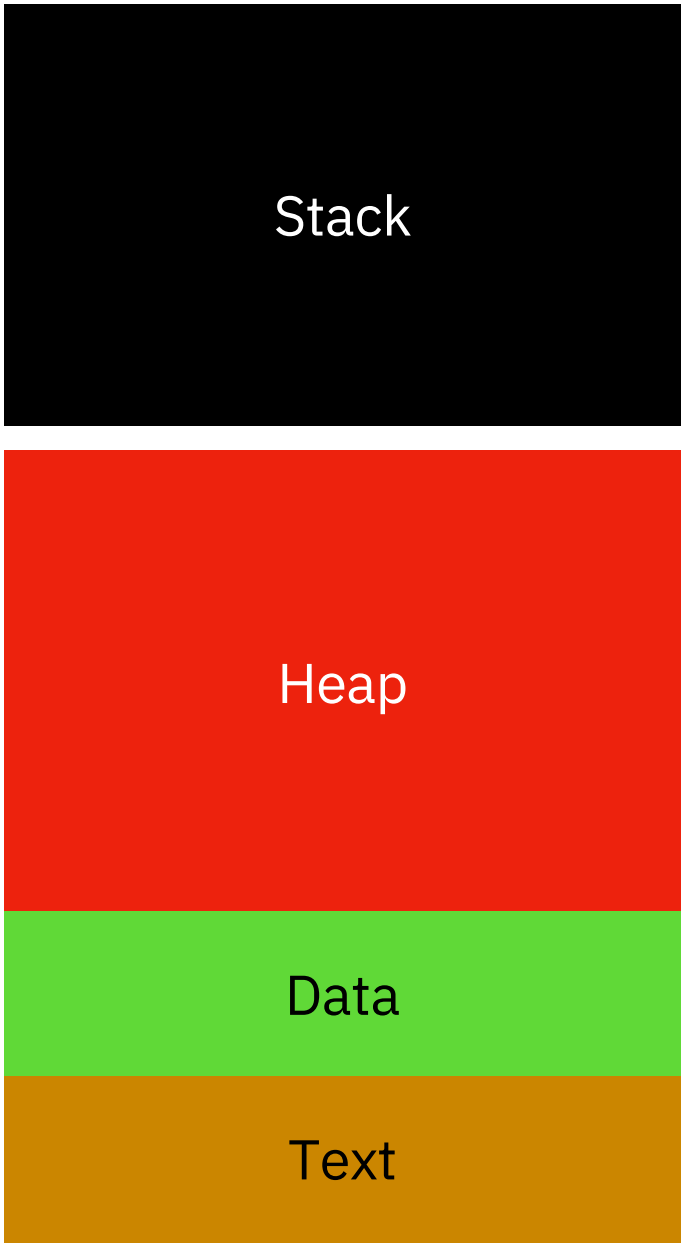



Fakultät für Informatik

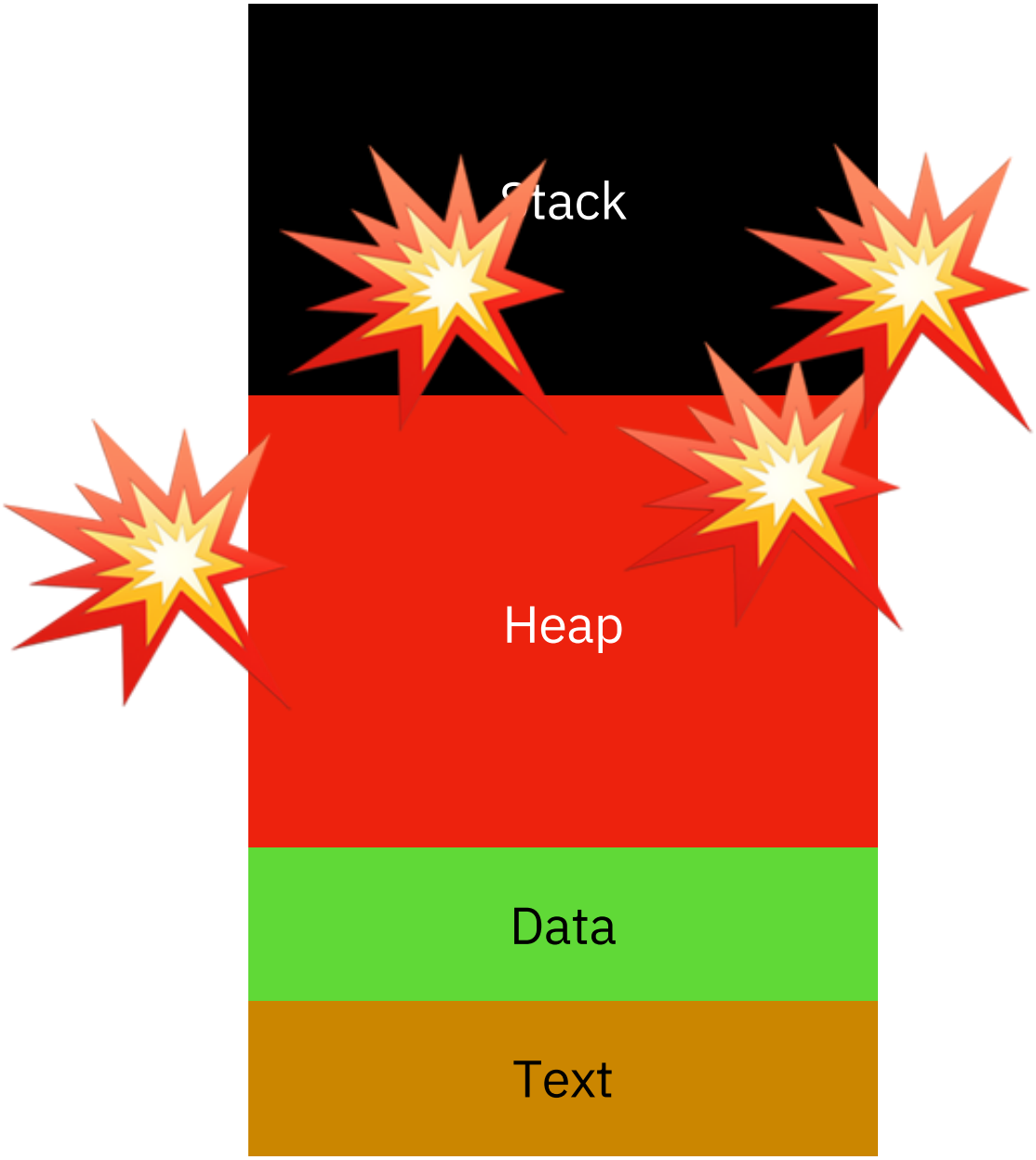
UDE

either by the modern C++ mechanisms or (old style) when the programmer decides to do so

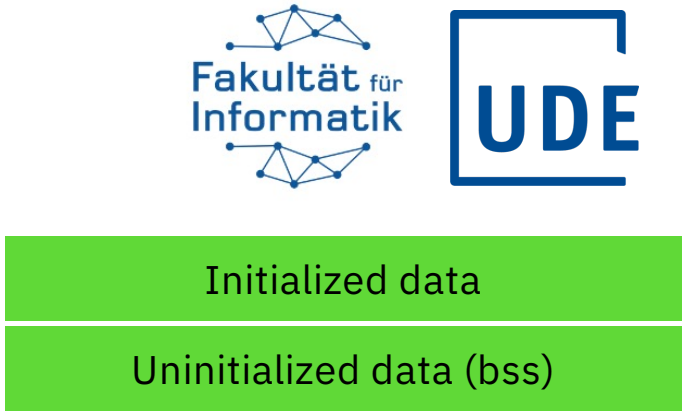
# *Stack and heap grow individually*



# *Out of memory*
# *Stack overflow*



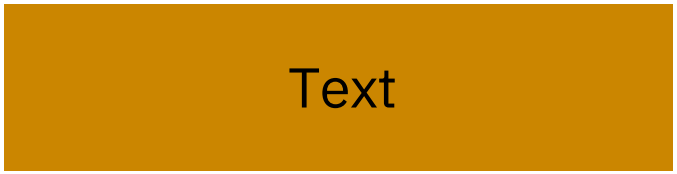
# *Memory Section: Data*




- Initialized data
  - all global, static, constant, extern variables that are initialized beforehand
- Uninitialized data (bss - block started by symbol)
  - all global and static variables that are not initialized beforehand and are set to 0 by default
  - Needs no memory in the executable, only reserved when program is launched

# *Memory section: Text*

- Does not contain text like strings etc. ;-)




- Contains machine code of compiled program
- Read only
- Can be shared between different programs to reduce memory usage

# Example

Fakultät für Informatik

UDE

bla.cpp

```
7   std::string s="FancyPants";//global variable stored in Initialized Data Segment in read-write area
8   char c;                    // Uninitialized variable stored in bss
9
10  int main()
11  {
12      static int i=1;        //static variable stored in Initialized Data Segment
13      static int j;          //Uninitialized static variable stored in bss
14      return 0;
15  }
```

# *Memory section: stack*

Fakultät für Informatik

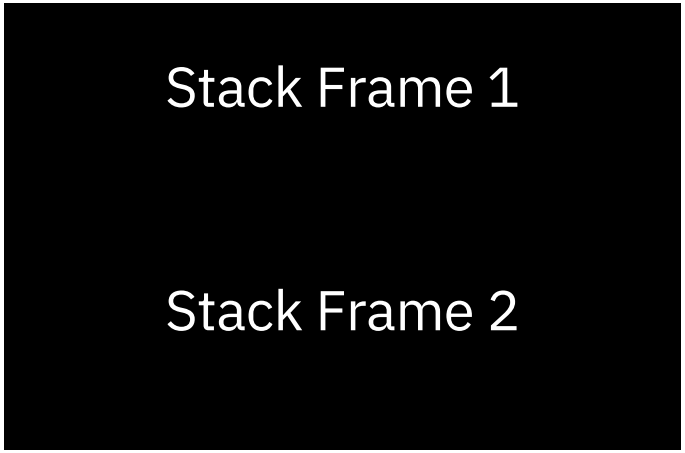
UDE

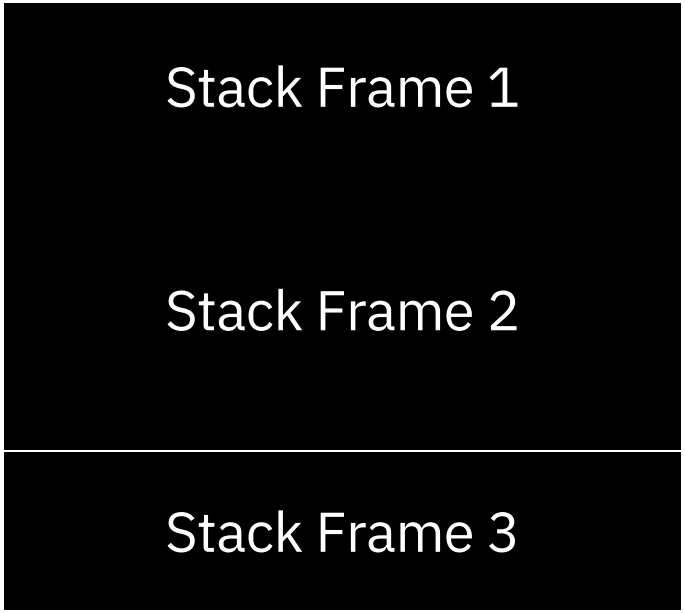
Stack

- The one you use most

- Contains local variables
- Contains passed arguments for a function/method
- Contains return address for what to do after the function is over
- Grows from high addresses down to low addresses

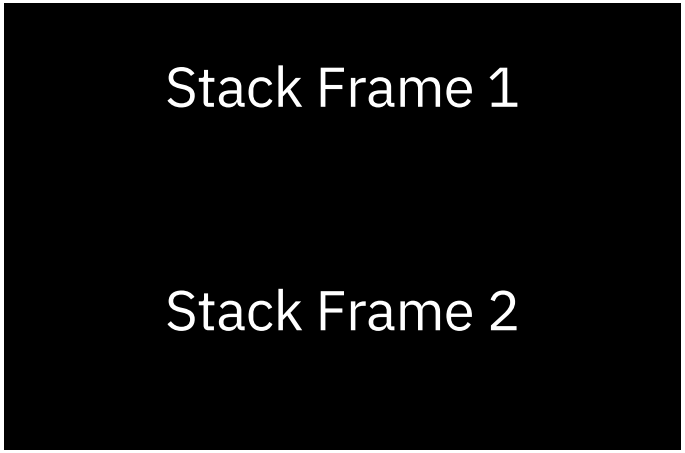
Stack Frame 1


Stack Frame 1
Stack Frame 2

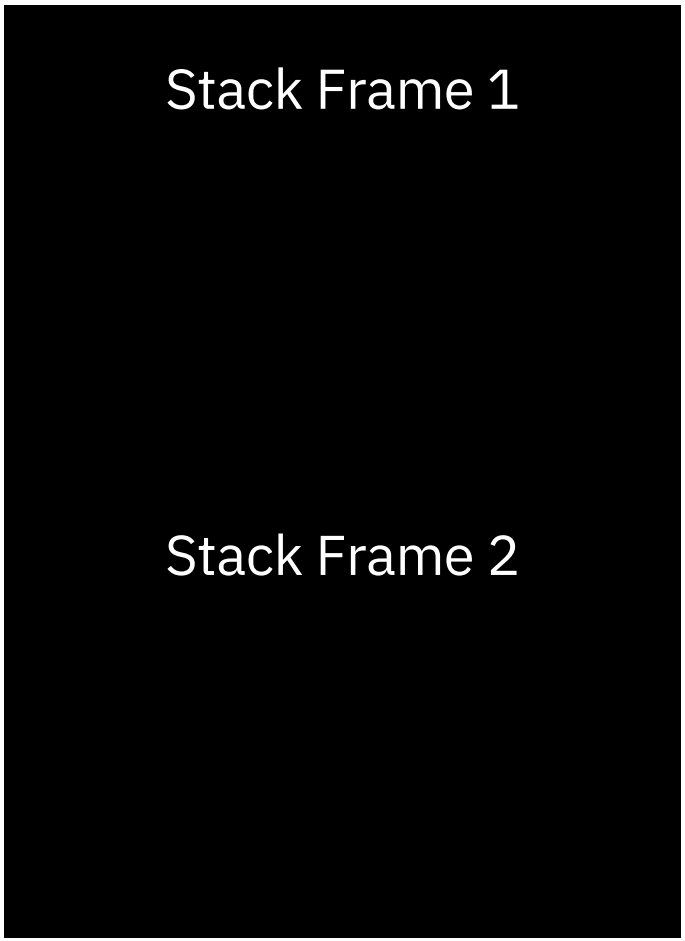



Fakultät für Informatik

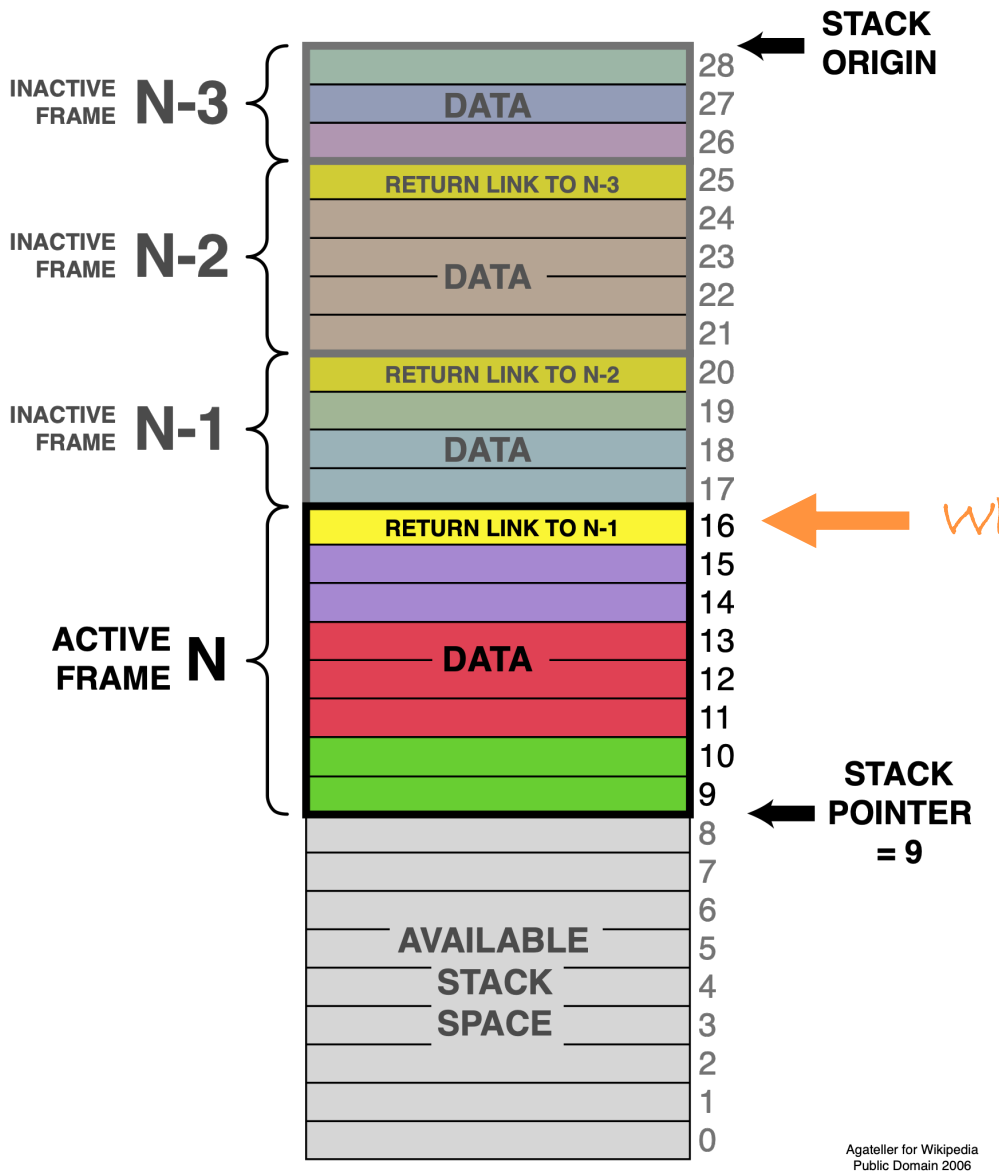
UDE


Stack Frame 1
Stack Frame 2

# Stack Frame 1


Stack Frame 1
Stack Frame 2

# *Stack*
## *- some details -*




What if we overwrite this?

Agateller for Wikipedia
Public Domain 2006

# *Stack smashing*
## *- Nowadays actually fixed -*

Fakultät für Informatik

UDE

- In old C/C++ it was easy to rewrite return addresses
- Just call new methods, even if the programmer did not plan it
- Surprise function call ;-)

- Nowadays the runtime checks this
- Your code still can have side effects

# *Never return references to stack-allocated data!*

Fakultät für Informatik

UDE

bla.cpp

```
7   int* create_array(){
8       int funkyArray[20];
9       return funkyArray;
10  }
```

Extremely dangerous: old C-style arrays!
Here we pass by reference without seeing it
(unless we know how old-style arrays are implemented)

**Straight to jail!**



# *Why is this a problem? And how can we do better?*

Fakultät für Informatik

UDE

- We know its 20 integers… Can't we return them by value?
  - In general: Yes

- Due to historical reasons we have a hidden return by pointer:

  `int funkyArray[20]` is the same as `int* funkyArray` to a memory region of 20 ints

- Modern solution:

  Use `std::array`

bla.cpp

```
7   int* create_array(){
8       int funkyArray[20];
9       return funkyArray;
10  }
```

# *Example*

*Please study it carefully (and understand what happens)!*

```
7   std::array<int,20> create_array(){
8       std::array<int,20> funkyArray;
9       return funkyArray;
10  }
11
12  int main(void) {
13      std::array<int,20> myArray = create_array();
14      for(int i = 0; i < 20; i++){
15          std::cout << myArray[i] <<std::endl;;
16      }
17      return 0;
18  }
```

```
> main.out
1
-183487768
1
-203358207
1
0
0
1841509216
1
-203357872
1
-203358184
1
644893
268435455
1841508856
1
-203358184
1
-203358032

Process finished with exit
code 0
```

Does the output surprise you?

# *Best practice: initialize memory*

Fakultät für Informatik

UDE

- Stack and heap memory is generally uninitialized
- Do not assume it is 0 (or anything else) upon allocation.

Not initializing memory is an invitation to Miss Randomness – and may lead to surprising behaviour. Finding bugs like this may give you hard times ;-)

# *Tiny but important change*

```
7   std::array<int,20> create_array(){
8       std::array<int,20> funkyArray = {} ;
9       return funkyArray;
10  }
11
12  int main(void) {
13      std::array<int,20> myArray = create_array();
14      for(int i = 0; i < 20; i++){
15          std::cout << myArray[i] <<std::endl;;
16      }
17      return 0;
18  }
```

```
> main.out
0
0
0
0
0
0
0
0
0
0
0
0
0
0
0
0
0
0
0
0

Process finished with exit
code 0
```

Better ;-)

# *Be aware of }*

Fakultät für Informatik

UDE

All stack allocated memory
is deallocated
at the end of a function call.

Local variables die when the execution of a function ends.

# *Where does your data go?*
## *- think of lifetimes -*

Fakultät für Informatik

UDE

- Text -> You don’t have too much access there

- Data -> Global/static variables
  - Only one copy per process! Side effects ***will*** happen
  - No need to clean up, data persists throughout whole execution

- Stack -> Data is gone after function call ends
  - Good for temporary variables
  - Automatically cleaned up at end of function call

# *One thing is missing*

Fakultät für Informatik

UDE

- The heap is for dynamically allocated memory for data that may persist for a longer time.



# *Dynamic chaos on the heap*

Fakultät für Informatik

UDE

The heap

- grows from lower addresses to high addresses
- contains dynamically allocated data (like the stack, but handles them differently).

What happens (and what does not happen)

- When a programme (that you write) requests memory, the heap reserves a region big enough for that.
- You have to care about freeing the memory.
- Memory on the heap is not protected from bad access.

# *How to care for the memory on the heap? (1)*

Fakultät für Informatik

UDE

- Old-style (as in C), very low level: `malloc/free`

- `malloc(n)` -> allocated n bytes on the heap and returns a pointer to that memory
- `free(ptr)` -> marks the pointer as freeable and the heap algorithm may reuse it

- If you use memory not granted by malloc, you will have segmentation faults
- If you forget to free memory that was allocated, you will have memory leaks

- Conclusion 1: If you use memory improperly, you will have a bad time.
- Conclusion 2: It is easy to use memory improperly with `malloc/free`

main.cpp

```
72  struct Thing{
73      void* memory;
74  };
75
76  void _init(Thing* bla, int size){
77      bla->memory = malloc(size);
78  }
79
80  void _delete(Thing** bla){
81          free((*bla)->memory);
82          free(*bla);
83  }
84
85  int main() {
86      Thing* t = (Thing *) malloc(sizeof(Thing));
87      _init(t, 50);
88      _delete(&t);
89      return 0;
90  }
```

We can implement some functions that allocate and free memory using malloc/free and we can call these from main()

```
72  struct Thing{
73      void* memory;
74  };
75
76  void _init(Thing* bla, int size){
77      bla->memory = malloc(size);
78  }
79
80  void _delete(Thing** bla){
81          free((*bla)->memory);
82          free(*bla);
83  }
84
85  int main() {
86      Thing* t = (Thing *) malloc(sizeof(Thing));
87      _init(t, 50);
88      _delete(&t);
89      _delete(&t);
90      _delete(&t);
91      return 0;
92  }
```



Noone stops us from calling them more than once and thereby crashing the programme.

```
❯ ./a.out
a.out(50095,0x1fcb73840) malloc: Double free of object 0x13b606120
a.out(50095,0x1fcb73840) malloc: *** set a breakpoint in malloc_error_break to debug
[1]    50095 abort      ./a.out
```

# *How to care for the memory on the heap? (2)*

Fakultät für Informatik

UDE

- Implement two functions `init` and `delete` for each user-defined datatype.
  - Initialize object, when object is created
  - Cleanup object, when object is freed

But:

- How to know that these should be called at the beginning/end?
- Any way to ***enforce*** that we use `init/delete`?
- Any way to make sure, we use them only once?

# *Constructor / Destructor*
## *- a key concept in object oriented programming -*

Fakultät für Informatik

UDE

- Constructors are executed when you initialize an object
- Destructors are executed when the object is cleaned up
- Both are special member functions / methods

- `New` -> Call constructor and store object ***on heap***
- `Delete` -> Call destructor and delete object ***on heap***

# *Example: constructor / destructor*

```
7    struct Thing{
8        void* memory;
9        Thing(int size){
10           this->memory = malloc(size);
11       }
12       ~Thing(){
13           free(this->memory);
14           this->memory = nullptr;
15       }
16   };
```

The constructor function has the same name as the datatype. It may have parameters.

The destructor has no parameters and an additional ~ as the first character of the name.

Note: In general, a constructor / destructor can do more than handling memory.
In this chapter we mainly talk about memory.

# *Using constructor / destructor – objects on stack*

- Two options to create an object ***on the stack***
  - Line 19/20 are almost identical
  - ➢ have a careful look at them ;-)
- No need to explicitly call the destructor
  - ➢ this happens automatically when the corresponding stack frame is left

```
7   struct Thing{
8       void* memory;
9       Thing(int size){
10          this->memory = malloc(size);
11      }
12      ~Thing(){
13          free(this->memory);
14          this->memory = nullptr;
15      }
16  };
17
18  int main(){
19      Thing t = Thing(50);
20      Thing t2(50);
21  }
```

# *Using constructor / destructor – objects on heap*

- new allocates object on the heap and calls constructor
- `delete` frees object on the heap and calls destructor

- Explicit destructor call necessary for heap allocation

- Always match every `new` with a `delete` to avoid memory leaks.

```
4   struct Thing{
5       void* memory;
6       Thing (int size){
7           this->memory = malloc(size);
8       }
9       ~Thing (){
10          free(memory);
11          this->memory = nullptr;
12      }
13  };
14
15  int main(){
16      Thing *t = new Thing(50);
17      delete t;
18      return 0;
19  }
```

# Objects on the heap vs. objects on the stack

```
72  struct Thing{
73      void* memory;
74
75      Thing(int size){
76          std::cout << "Constructor " << size << std::endl;
77          this->memory = malloc(size);
78      }
79      ~Thing(){
80          std::cout << "Destructor "  << std::endl;
81          delete memory;
82          this->memory = nullptr;
83      }
84  };
85
86  int main() {
87      Thing* t = new Thing(50);
88      Thing t2(30);
89      delete t;
90      return 0;
91  }
```

```
> main.out
Constructor 50
Constructor 30
Destructor
Destructor

Process finished
with exit code 0
```

create t on the heap using new

create t2 on the stack

explicitly remove t from the heap using delete

t2 is automatically deleted when the main function finishes

# *Conclusion – memory on the heap*

Fakultät für Informatik

UDE

Manually managing heap allocated memory is hard!

# *Avoiding memory leaks*

Fakultät für Informatik

UDE

- Assumption:  All objects in the heap that we still need, must be bound to a variable.

- Reason: if there is no variable (no pointer) connected to an object we loose access to it and create a memory leak, i.e. a memory region that cannot be used any more for the current process.

# *Binding*

Variable + Data (Object)



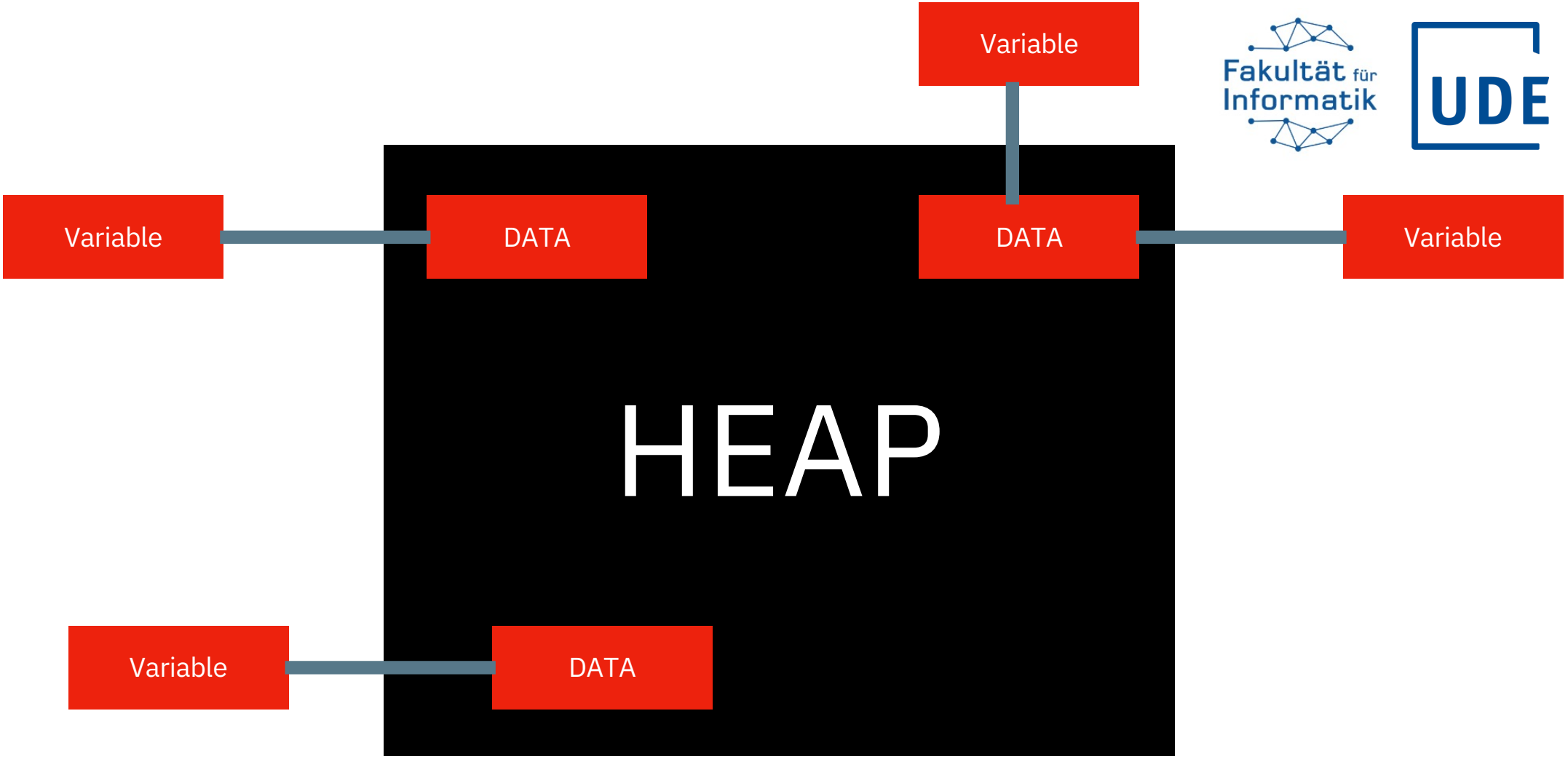
# *Short excursion: Garbage collectors*
## *- what other programming languages do -*

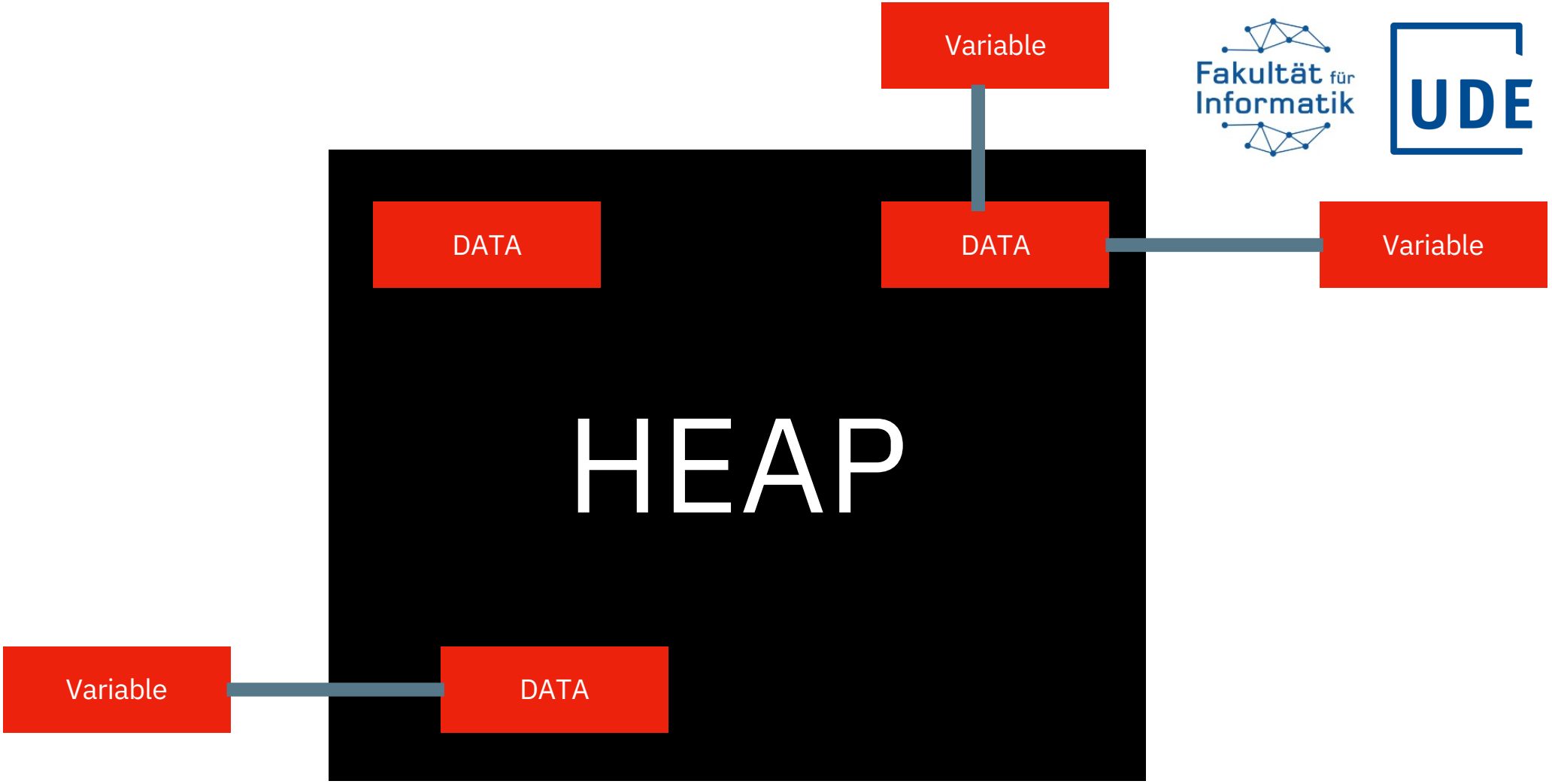
Fakultät für Informatik

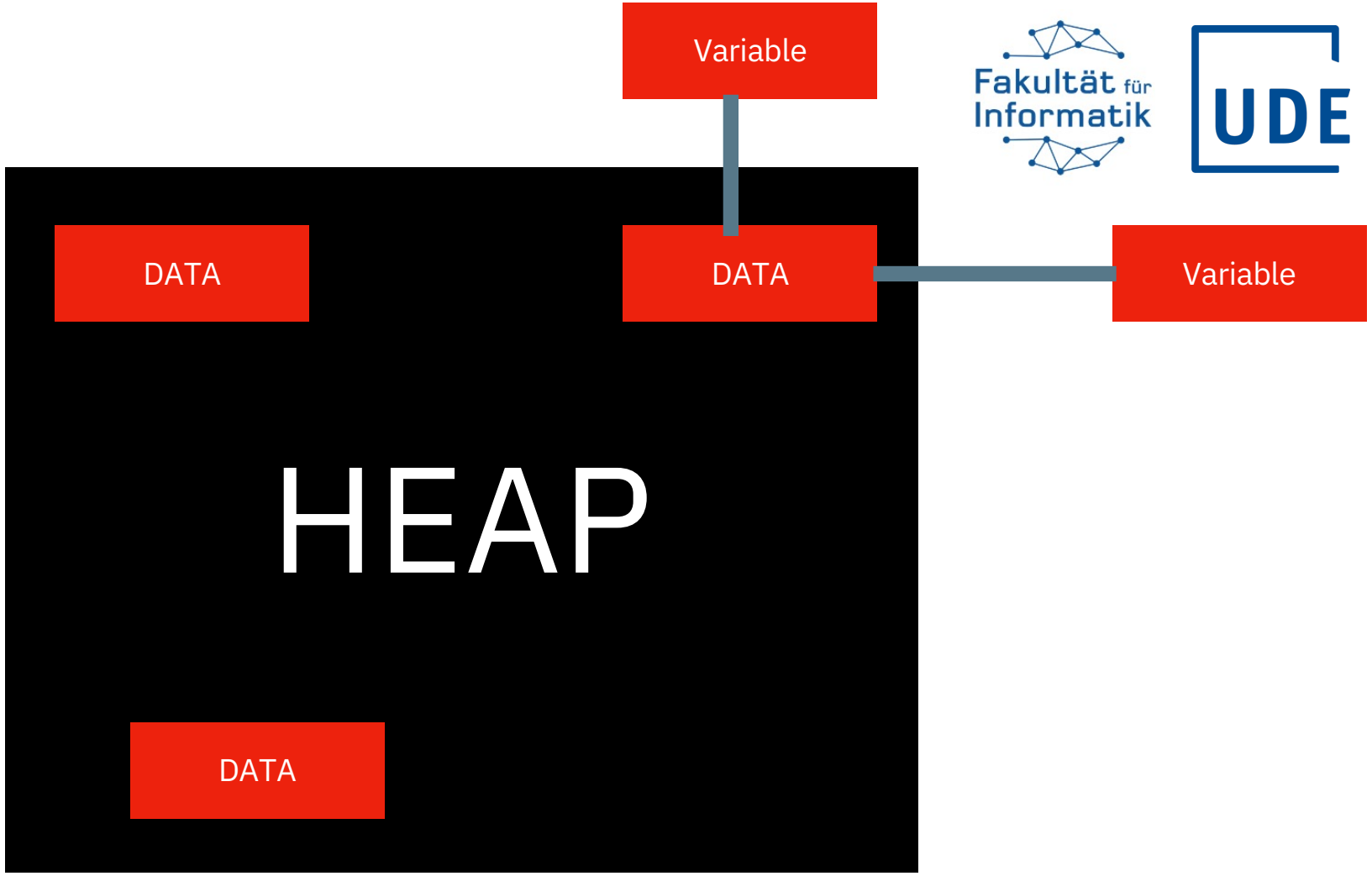
UDE

- Allocate all non-fundamental data types on the heap
- Regularly stop the world and check which objects are reachable
- Delete all objects that are no longer reachable

- Spoiler alert: High performance penalty - we don’t want that ;-)


Variable
Variable
DATA
DATA
Variable
HEAP
Variable
DATA


Variable
DATA
DATA
Variable
HEAP
Variable
DATA


Variable
DATA
DATA
Variable
HEAP
DATA

# *Garbage Collection Time*




HEAP
DATA
Variable

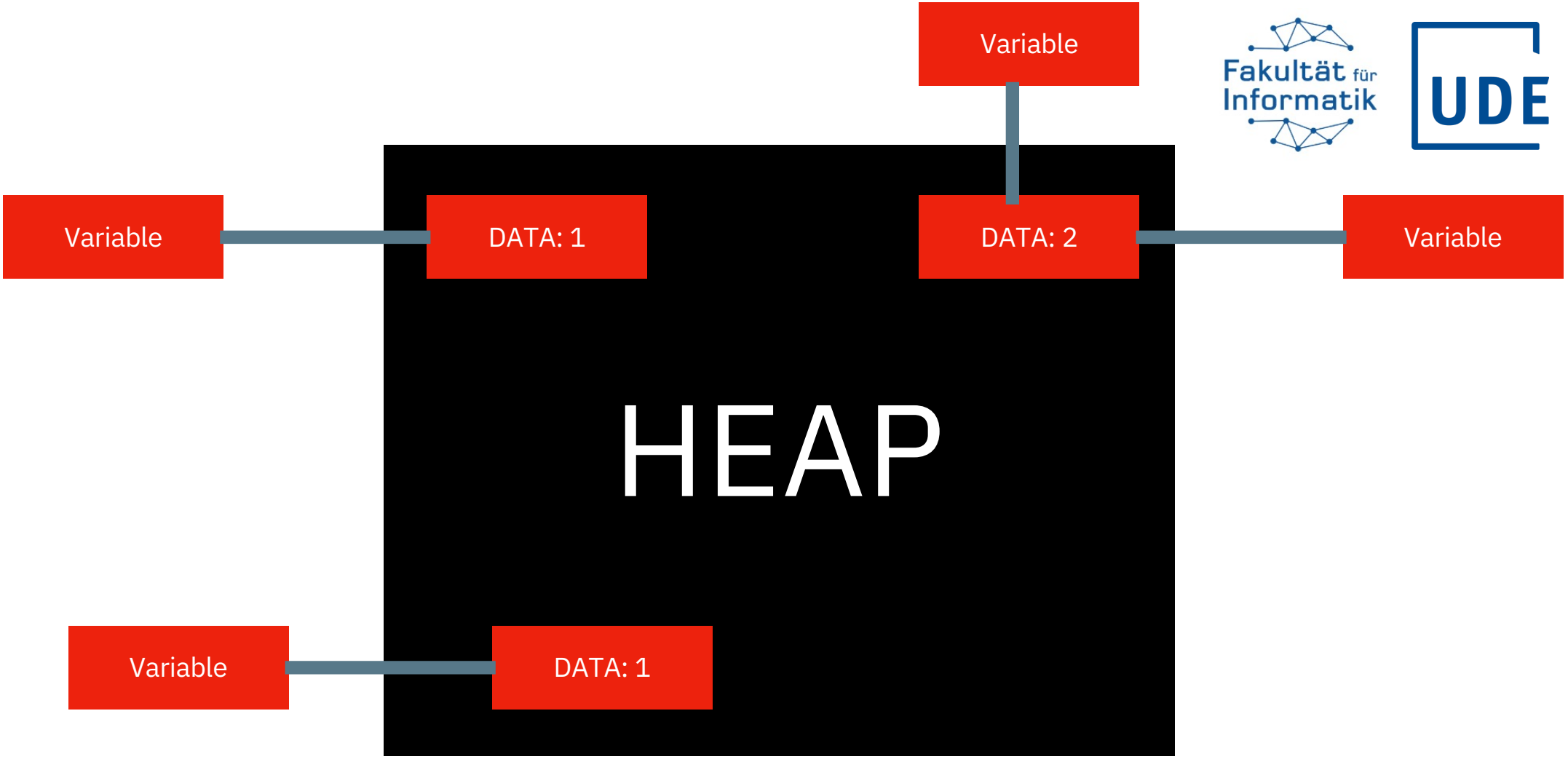
# *Drawback*

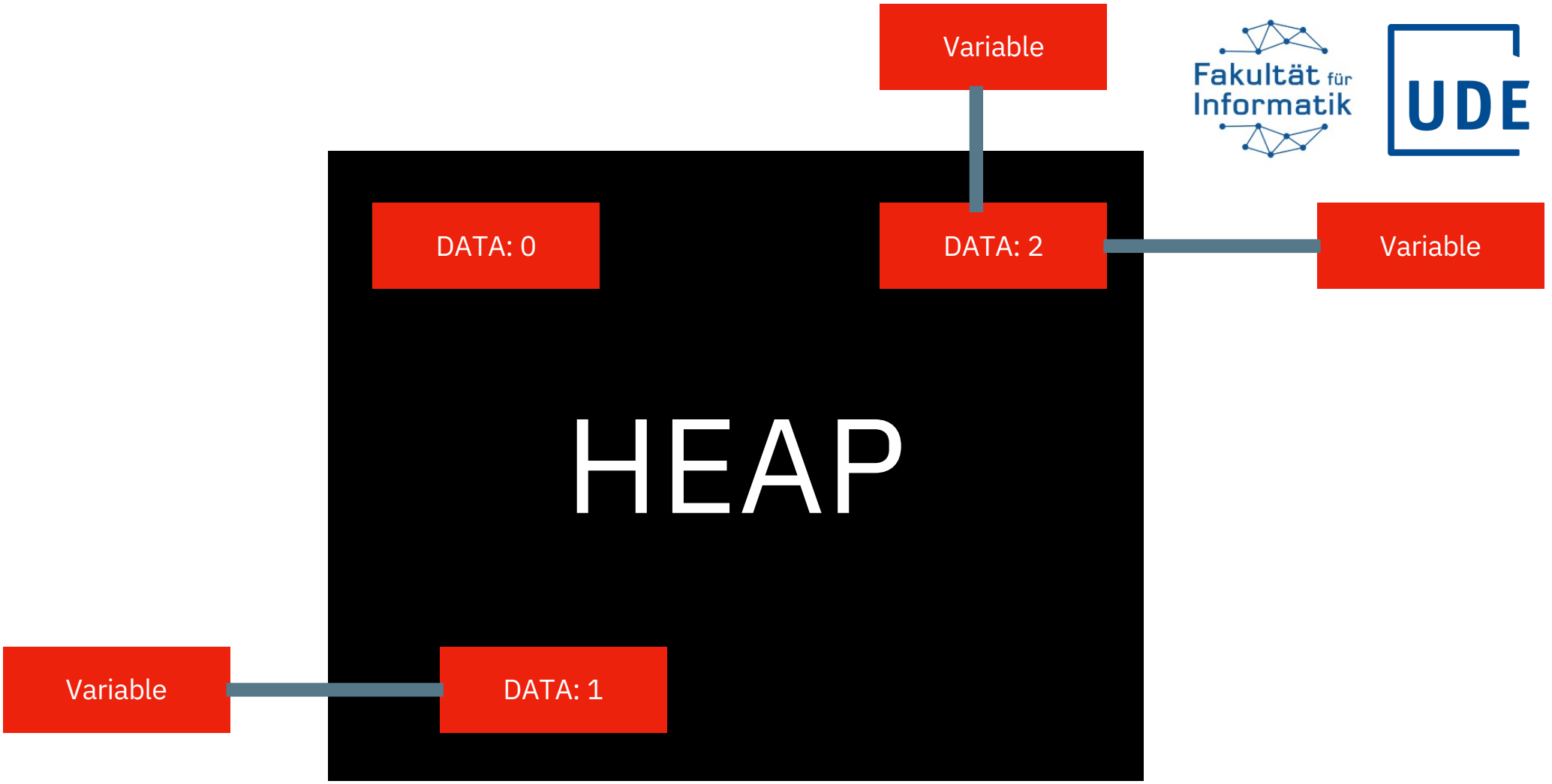
Stopping the world for garbage collection considerably slows the execution.

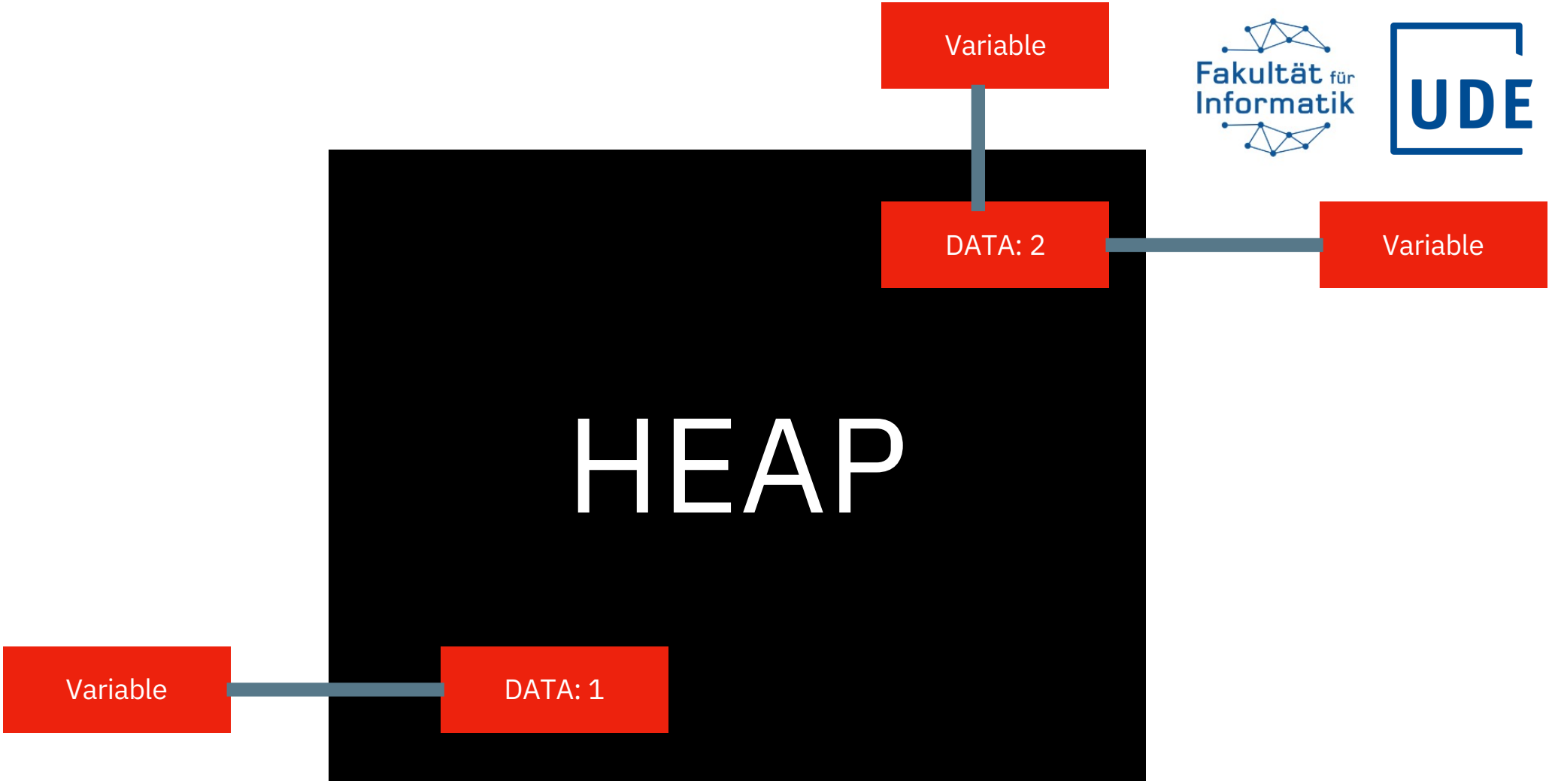
# *Reference counting*

- Count for each heap allocated data, how many variables reference it
- Increase counter for each new variable
- Decrease counter for each variable that is freed

- tracking, if we still need a certain object (on the heap)


Variable
Variable
DATA: 1
DATA: 2
Variable
HEAP
Variable
DATA: 1


Variable
Fakultät für Informatik
UDE
DATA: 0
DATA: 2
Variable
HEAP
Variable
DATA: 1


Variable
Fakultät für Informatik
UDE
DATA: 2
Variable
HEAP
Variable
DATA: 1


HEAP
DATA: 1
Variable
Variable
DATA: 1


DATA: 1
Variable
HEAP
DATA: 0


HEAP
DATA: 1
Variable

Fakultät für Informatik

UDE



Fakultät für Informatik

UDE

# HEAP

# *Outlook (for next week): Smart pointers*
## *– the modern C++ concept to apply rules to pointers –*







- `unique_ptr` and `shared_ptr` as ways to simplify memory management
- `unique_ptr`
  - sole owner of data, freed when variable goes out of scope
- `shared_ptr`
  - Reference counted, freed when last variable that references this memory goes out of scope

- Using raw pointers (like in C) is a good idea if you want to learn how valgrind works
  - Valgrind is a tool to debug memory management bugs/errors

# *Summary for today: Memory sections*

Fakultät für Informatik

UDE

- Memory section: Text
  - machine code
- Memory section: Data
  - global and static variables
- Memory section: Heap
  - dynamic memory for objects that exist independent of the programme flow
  - ➢ most likely the trickiest part regarding memory management
- Memory section: Stack
  - dynamic memory for local variables and function calls

Fakultät für Informatik

# Questions?



"live" in lecture and exercise



discussion forum @ Moodle (preferred, everyone can reply)



by e-mail (use your @uni-due.de email address!)